

Bankchain — Presentación y Runbook

Nodo de ledger inmutable de Ciudadan · Tokens **LABORY** y **CIT** · API v1 (puerto 6633)

1. Qué es este sistema (discurso de 30 segundos)

Es el **libro contable digital e inmutable** de Ciudadan. Registra cada peso electrónico que se mueve: saldos, subsidios, pagos y los tokens de inversión CIT. Toda operación queda firmada, encadenada por hash en bloques y guardada en SQLite. Nadie puede borrar ni falsificar un movimiento ya registrado. No hay minería ni red: es **un solo nodo autoritativo** que corre nuestro equipo; los usuarios solo consultan por API y firman con su llave privada.

2. Quién es quién

Rol	Qué es
Nodo	Única máquina que mina y firma bloques (index.js en localhost:6633)
Admin	Tú/operador: tiene ADMIN_TOKEN para emitir, registrar agencias, pagos
Cliente	Usuario final con su par de llaves; firma sus transacciones
Agencia	Entidad registrada con nivel_subsidio (taxis, negocios del marketplace)
Tesorería	Cuenta del sistema 0x...0001: paga rendimientos y payouts
Reserva	Cuenta del sistema 0x...0002: guarda el principal de CIT

Llave → **dirección**: llave privada (secreto) → llave pública (hex) → dirección 0x + sha256(pública).slice(-40). Firmar demuestra que tienes la llave **sin revelarla** (secp256k1).

3. Avances por fases

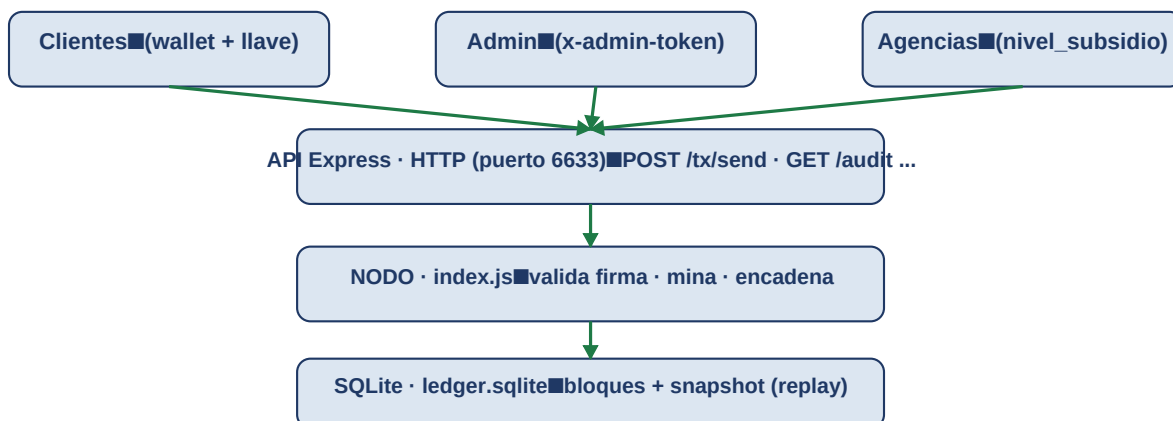
Fase	Qué se logró
0	Contrato de API v1 congelado (CONTRATO_API.md): formato de firma, endpoints, códigos de error
1	Ledger + nodo + minería + persistencia SQLite con replay (GET /audit)
2	Transacciones firmadas con nonce anti-replay y validación de saldo
3	Subsidios: pago compartido usuario + agencia en un bloque atómico
4	earn por tarea/anuncio, idempotente por origin_id (no se paga dos veces)
5	Tokens CIT (bloquear, rendimientos, madurez), sync de agencias desde Strapi, export del ledger

Fase	Qué se logró
Extras	Seguridad (rate-limit, token admin, firma), 59 tests, guías y mini-cliente

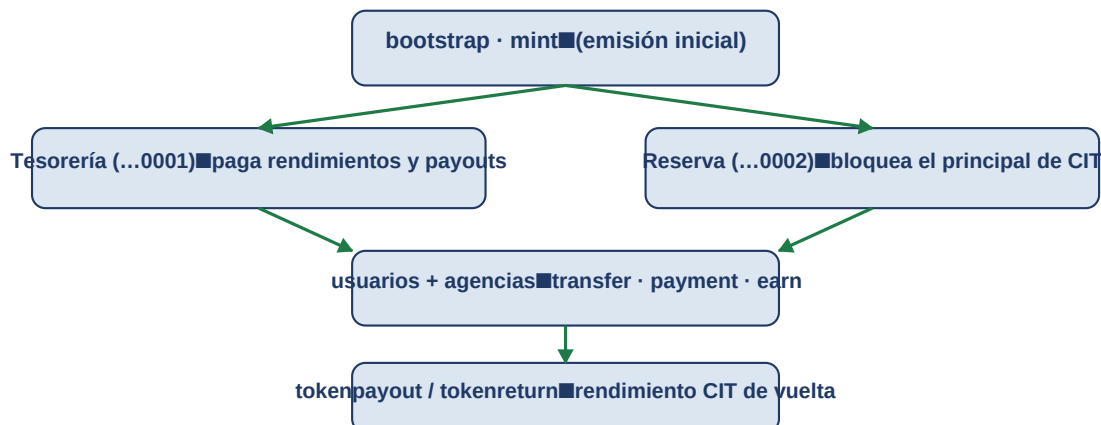
Las transacciones que el ledger registra: **mint**, **transfer**, **reward**, **payment**, **earn**, **tokenlock**, **tokenpayout**, **tokenreturn**, **payout**.

4. Cómo funciona el sistema (diagramas)

Arquitectura: los clientes llegan por HTTP; el nodo es el único que valida, firma y encadena; todo persiste en SQLite.



Ciclo de dinero: emisión → tesorería/reserva → usuarios y agencias → rendimientos CIT de vuelta.



5. ¿En qué momento entra Strapi?

Strapi es la **capa administrativa** (catálogo, configuración, backoffice); la blockchain es la **única fuente de verdad** de balances y transacciones. Strapi nunca guarda private keys ni ledger.

- **Registro de agencias** — las agencias (taxis, negocios del marketplace) se administran en Strapi y el nodo las sincroniza por **POST /agencias/sync**, que consume `{STRAPI_URL}/api/agencias` con Bearer token y mapea `wallet_address + nivel_subsidio`.
- **En la demo** — el paso 8 de `npm run demo` es justo el sync; sin STRAPI_URL responde **502 STRAPI_URL_REQUIRED**, momento ideal para decir: "aquí se conectaría Strapi cuando exista".
- **Límites** — el nodo no confía en Strapi para balances; solo usa el registro de agencias.

6. Cómo echarlo a andar

Arranque del nodo (terminal 1):

```
cd ciudadan_bankchain
```

```

npm install          # solo la primera vez
npm run bootstrap    # siembra 100,000 LABORY + 50,000 CIT al nodo
npm start            # arranca el nodo en localhost:6633

```

Verificar que está vivo (terminal 2):

```

curl localhost:6633/health
# {"success":true,"data":{"status":"ok","mode":"single-node",...}}

```

7. Comandos de referencia rápida

```

npm start            # arranca el nodo
npm run bootstrap    # siembra saldos iniciales del nodo
npm run demo         # demo guiada de las 5 fases (requiere nodo arriba)
npm run smoke        # smoke test e2e
npm run client       # mini-cliente (new/list/balances/wallet/tx/send)
npm test             # 59 tests
npm run reset        # borra el ledger (con -- --wallet también la wallet del nodo)

```

Mini-cliente del usuario final (firma por ti):

```

npm run client new carlos      # crea wallet: imprime address + privateKey
npm run client list           # wallets guardadas
npm run client balances carlos # saldos por token
npm run client wallet carlos LABORY
npm run client send carlos maria 25 LABORY # firma y envía
npm run client tx carlos      # historial de la wallet

```

8. Demo en vivo sugerida (5 min)

Flujo completo con un ledger limpio desde cero:

```

kill -f "node index.js" 2>/dev/null # asegura que no haya nodo viejo
npm run reset -- --wallet            # ledger limpio desde cero
npm run bootstrap                    # saldos iniciales
npm start                            # (terminal 1)
npm run demo                         # (terminal 2): las 5 fases
npm run client                       # (terminal 2): transferencia firmada

```

Pasos que causan mayor impacto:

- **GET /audit** — reproduce todo el ledger desde cero y lo compara con el estado: demuestra la integridad.
- **send firmado** entre dos wallets — el usuario firma localmente y el nodo verifica firma, nonce y saldo.
- **Pago con subsidio** — usuario paga 15 y la agencia 45, el destino recibe 60 en un solo bloque atómico.

9. Preguntas previsibles y respuestas

Pregunta	Respuesta corta
¿Por qué blockchain si es un solo nodo?	La inmutabilidad no requiere red: el encadenamiento por hash y las firmas bastan. Una red p2p no aporta a esta escala.
¿Es dinero real?	No: son laborys electrónicos (LABORY), moneda interna. El nodo solo registra saldos, no toca dinero físico.
¿Qué pasa si se cae el nodo?	El ledger está persistido en SQLite y se exporta (GET /ledger/export); al arrancar se reanuda desde el último bloque.
¿Cómo se evita el fraude?	Firma secp256k1 + nonce anti-replay + saldo: nadie falsifica ni reenvía una transacción.

Más detalle en: [GUIA-DESDE-CERO.md](#), [CONTRATO_API.md](#) y [SECURITY.md](#)